Fiery Image Transfer Service Specification

Fiery DFE-Press Data Path Interface



Revision History

Version	Change Log	Date	Author
1.0	Initial Draft	01/10/2020	Aloke Bordia

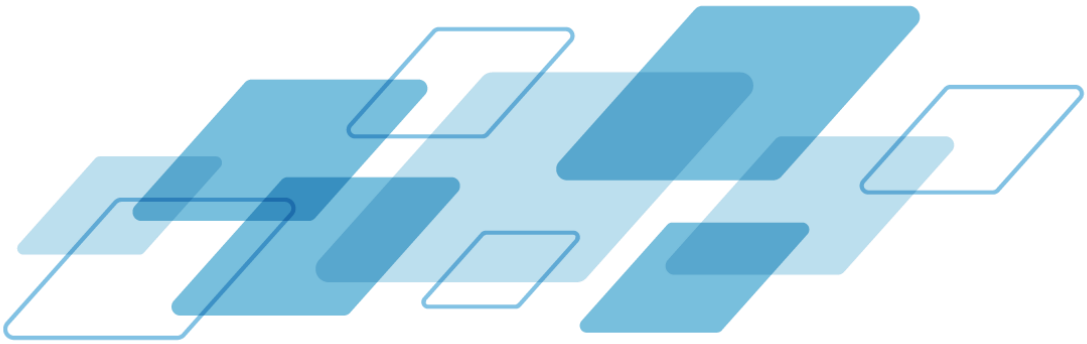




Table of Contents

1 OVERVIEW 1

1.1 CONTROL PATH 1
1.2 DATA PATH 1

2 FITS 2

2.1 PROTOBUF 2
2.2 CLIENT-SERVER 2
2.3 DATA FORMAT 2

3 SEQUENCES 3

3.1 STARTUP 3
3.1.1 Configuration 3
3.2 PRINTING 4
3.2.1 Print a Job 4

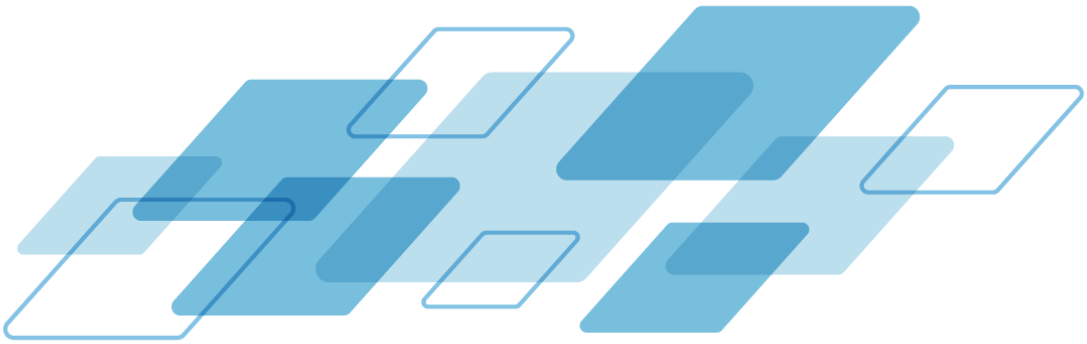
4 PROTOCOL API 5

4.1 PROTO FILE 5
4.2 HEADER (C/C++ ONLY) 7

5 SAMPLE CODE 8

5.1 CONFIGURATION 8
5.1.1 Sending CLIENT_CONNECTED 8
5.1.2 Receiving SERVER_CONFIG Header 8
5.1.3 Receiving SERVER_CONFIG Data 9
5.2 PRINTING 10
5.2.1 Receiving SERVER_DATA Header 10
5.2.2 Receiving SERVER_DATA Ink Data 10

6 PACKAGE 12



1 Overview

This document is intended to serve as a reference for the Printer Controller Data communication interface between Fiery DFE (Digital Front End) and a printer. Communication between the DFE (i.e. server) and printer (i.e. client) is split into two categories:

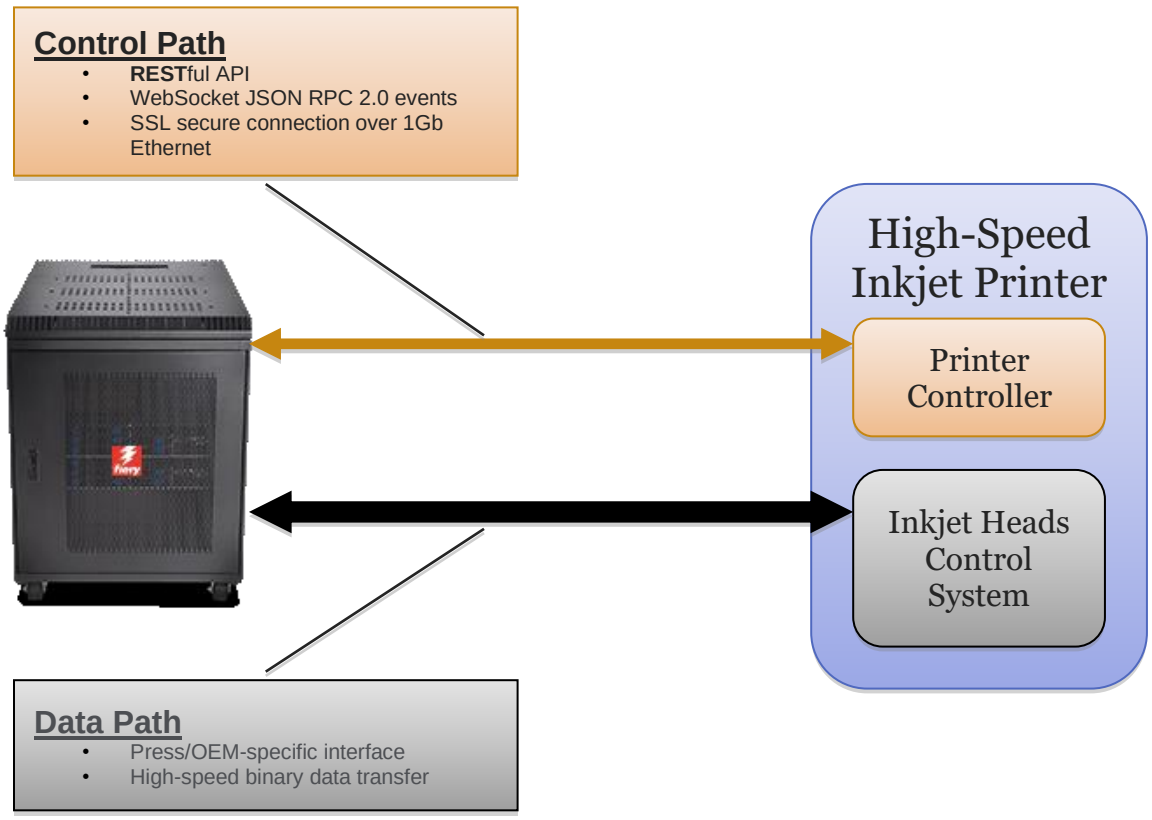
- Control Path
- Data Path

1.1 Control Path

The Control Path is the interface for handling all management/control functions. It is discussed in a separate document.

1.2 Data Path

The Data Path is the interface for handling the transfer of image/ink data from the DFE to the printer. The Control Path, by receiving events and acknowledging them using the RESTful API, allows data to enter the Data Path. This document discusses the details for Data Path, referenced as FieryPath Image Transfer Service (FITS) for the rest of this document.



2 FITS

This section provides an overview of Data Path, also known as FITS (FieryPath Image Transfer Service). For the purposes of this document, it is assumed that 10GB Ethernet interface will be used for FITS.

2.1 Protobuf

Google Protocol Buffer v2.50 (henceforth referred to as protobuf) is used for FITS API definition. More details for protobuf can be found at https://developers.google.com/protocol-buffers/docs/reference/proto2-spec.

Printer client can be written in any language supported by protobuf v2.50 (C/C++, Java, Python, Go).
https://developers.google.com/protocol-buffers/docs/proto

All examples shown in this document use C/C++. You can download the header and library files from https://github.com/protocolbuffers/protobuf/tree/v2.5.0. **Please download v2.50 to be compatible with FITS**.

2.2 Client-Server

- FITS will act as a set of TCP servers
- At startup, the printer should connect to FITS configuration server at the well-known IP address using port 9210
- The printer should do that for all Data Path Ethernet interfaces
- Once connected, DFE will send the list of inks for that interface along with the port to connect to receive ink data
- The printer should then connect to each individual ink at the specified port on that interface IP address

At this point, the printer is ready to receive image data for all configured inks.

2.3 Data Format

- Image data will be full-frame halftoned planar format
- Printer will receive data for each ink on a separate TCP socket connection
- Data will contain information about pixels, resolution, bits per pixel, bytes per scanline
- Printer is expected to always read all data sent by the server even in the cases when a job is canceled or during error recovery process

All data sent over the sockets will be in Little Endian Byte Order.

3 Sequences

This section describes the main sequences for FITS. It is broken up into two categories: Startup and Printing.

3.1 Startup

3.1.1 Configuration

The Configuration sequence describes the steps the printer client must execute to connect to the FITS server and get configuration for each ink. The client should connect to each IP address on the well-known port 9210 and send CLIENT_CONNECTED message. The loop in this sequence indicates multiple such connections that the printer client may need to initiate depending on the number of Ethernet interfaces (n).

Once the client connects, FITS server will send information about the inks configured on that Ethernet interface along with the port to use to receive ink data during printing. As seen from the sequence diagram, the client will first receive 4 bytes of data (FITS_HEADER_LENGTH) which will indicate the size of the common header. This header (COMMON_HEADER) will contain information about the inks configured on that Ethernet interface along with the port (SERVER_CONFIG).

The printer client should now initiate a socket connection for each of the configured inks (m) on the specified port. At this point, the printer client is ready to receive image data for each ink once printing is initiated.

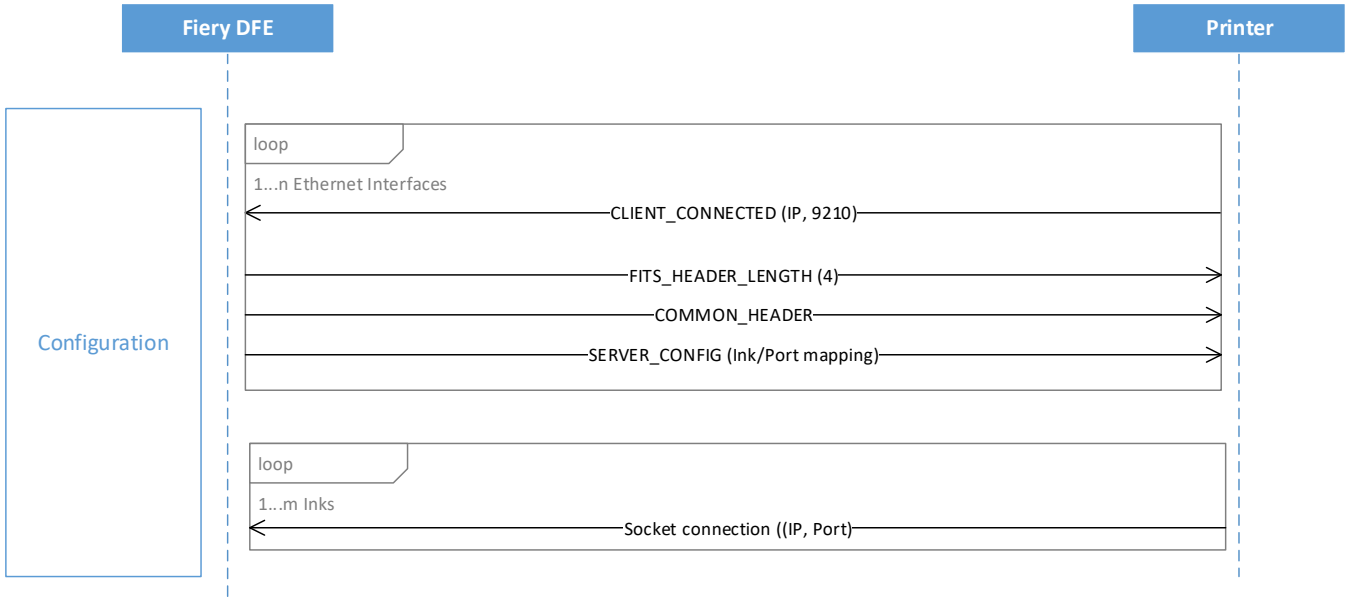


Figure 1: Configuration Sequence

3.2 Printing

3.2.1 Print a Job

The Print a Job sequence describes the steps the printer client must execute to successfully print a job. Once printing is initiated, the client will receive image data for each configured and connected ink (m). As seen from the sequence diagram, the client will first receive 4 bytes of data (FITS_HEADER_LENGTH) which will indicate the size of the common header. This header (COMMON_HEADER) will contain information about the image data. The printer client should use this information to receive SERVER_DATA for all configured inks and successfully print a job.

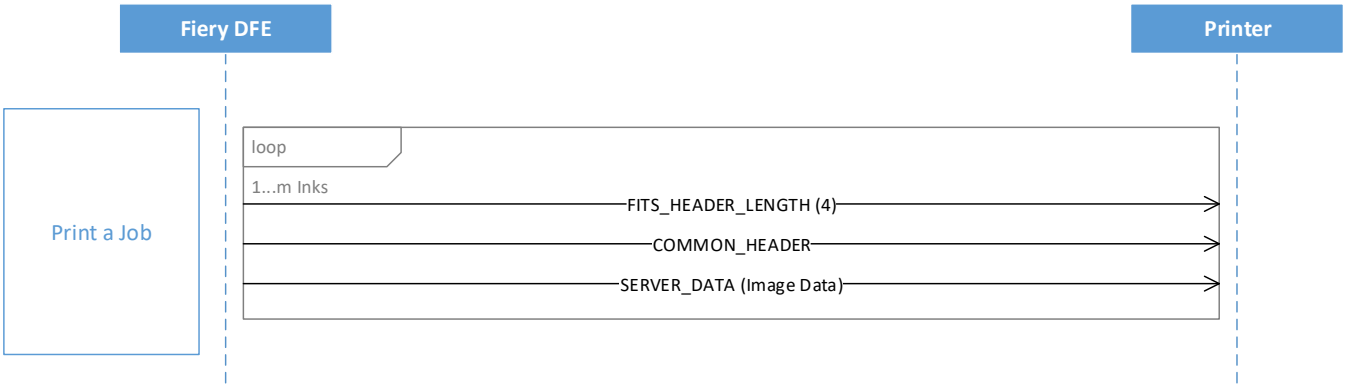


Figure 2: Print a Job Sequence

4 Protocol API

4.1 Proto File

The proto file (fieryImageTransferServiceIfc.proto) contains the Google Protocol Buffer FITS API. As indicated earlier, the printer client can be created in any language supported by protobuf v2.5.0.

```
// Protocol definition for Fiery Image Transfer Service (FITS) client/server.

package EFI.FITS.CONSTANT;

// Messages used between FITS client and server. Message with prefix CLIENT_ is initiated by the
client, ones with prefix SERVER_ is initiated by the server
enum MESSAGE_TYPE
{
   CLIENT_CONNECTED  =   0;  // Sent by client at startup
   SERVER_CONFIG     =   1;  // Sent by server in response to CLIENT_CONNECTED
   SERVER_DATA       =   2;  // Sent by server during printing
}

enum TAG
{
   FITS = 0x70738483;   // "FITS", used to validate socket integrity
}

enum VERSION
{
   VERSION_1 = 1;    // Version information
}

message COMMON
{
   message PORTS
   {
      required string   ink    = 1;   // Configured ink name
      required uint32   port   = 2;   // Port to connect to receive data for this ink
   }

   message MAPPINGS
   {
      required uint32   number_of_inks    = 1;   // Number of inks on this interface
      repeated PORTS    ink_ports         = 2;   // Ink/port mapping
   }

   // Dimension used in other messages
   message DIMENSION
   {
      required uint64   x = 1;
      required uint64   y = 2;
   }

   // COMMON_HEADER used for all messages
   message HEADER
   {
      required TAG            tag                  = 1;
      required VERSION        version              = 2;
      required MESSAGE_TYPE   message_type         = 3;
```

```
      required uint32         message_size_in_bytes = 4;
   }

   // Contains image data
   message IMAGE
   {
      required string        id                = 1;
      required string        ink               = 2;
      required uint32        bits_per_pixel    = 3;
      required uint32        bytes_per_scanline = 4;
      required DIMENSION     pixels            = 5;
      required DIMENSION     resolution        = 6;
      required uint64        size_in_bytes     = 7;
   }

   // Data for a Job during printing
   message JOB
   {
      required string   id    = 1;
      required IMAGE    image = 2;
   }
}

message MESSAGE
{
   message SERVER_CONFIG
   {
      required COMMON.MAPPINGS inks = 1;
   }

   message SERVER_DATA
   {
      required COMMON.JOB      job = 1;
   }
}
```

4.2 Header (C/C++ only)

As indicated earlier, FITS server will send 4 bytes (FITS_HEADER_LENGTH) before sending the COMMON_HEADER information. For a C/C++ client, fieryImageTransferServiceIfc.h file shows this information.

```
#ifndef __FIERY_IMAGE_TRANSFER_SERVICE_IFC__
#define __FIERY_IMAGE_TRANSFER_SERVICE_IFC__

#include <cstdint>

#define FITS_HEADER_LENGTH 4    // Length in bytes of initial FITS header

// This struct defines a size member which will contain the actual size of the header sent
(COMMON_HEADER). This is defined in fieryImageTrasferServiceIfc.proto
typedef struct
{
    uint8_t size[FITS_HEADER_LENGTH];
} FITS_HEADER_SIZE;


#endif
```

5 Sample Code

All the sample code examples shown here are in C/C++. It is pseudo code.

5.1 Configuration

5.1.1 Sending CLIENT_CONNECTED

```
// Create the socket configuration client at well-known IP Address and Port
Socket* socket_config_client = new Socket(ip, 9210);

COMMON_HEADER header;
header.set_tag(FITS);
header.set_version(VERSION_1);
header.set_message_type(CLIENT_CONNECTED);
header.set_message_size_in_bytes(0);
int size = header.ByteSize();

// Serialize and send the data
if (header.SerializeToArray(buffer, size))
{
    socket_config_client->Send(buffer, size);
}
```

5.1.2 Receiving SERVER_CONFIG Header

```
// First get 4 bytes (FITS_HEADER_LENGTH)
unsigned int size = 0;
memcpy(&size, p_void, FITS_HEADER_LENGTH); // p_void is receive socket buffer

// Get the header based on the size received above
int ret = socket_config_client->Receive(buffer, size);
if (ret > 0 && ret == size)
{
    // Parse the common header
    COMMON_HEADER header;
    header.ParseFromArray(buffer, size);
    if (header.has_tag() && header.has_version() && header.has_message_type() &&
          header.has_message_size_in_bytes())
    {
        TAG tag_received = header.tag();
        VERSION version_received = header.version();
        MESSAGE_TYPE message_type_received = header.message_type();
        unsigned int message_size_in_bytes_received = header.message_size_in_bytes();

        // Now receive the server configuration
        if (message_size_in_bytes_received != 0 && message_type_received == SERVER_CONFIG)
            ReceiveServerConfigData(message_size_in_bytes_received); // See 5.1.3
        else
            LOG(ERROR) << "Incorrect config message_size_in_bytes or message_type received";
    }
    else
        LOG(ERROR) << "Incorrect config header received, does not have all elements";
}
else
    LOG(ERROR) << "Incorrect fixed size header received, ret=" << ret << " size=" << size;
```

5.1.3 Receiving SERVER_CONFIG Data

This is the ReceiveServerConfigData function shown in 5.1.2

```
int ret = socket_config_client->Recieve(buffer, message_size_in_bytes_received);
if (ret > 0 && ret == message_size_in_bytes_received)
{
    MESSAGE_SERVER_CONFIG message;
    message.ParseFromArray(buffer, message_size_in_bytes_received);
    if (message.has_inks())
    {
        COMMON_MAPPINGS mappings = message.inks();
        if (mappings.has_number_of_inks())
        {
            unsigned int numInks = mappings.number_of_inks();
            for (unsigned int n = 0; n < numInks; n++)
            {
                EFI::FITS::CONSTANT::COMMON_PORTS ports = mappings.ink_ports(n);
                std::string ink = ports.ink();
                unsigned int port = ports.port();

                // Create the socket client for each ink at this port
                Socket* socket_data_client = new Socket(ip, port, ink);
            }
            else
                LOG(ERROR) << "Incoming server config message does not have inks";
        }
        else
            LOG(ERROR) << "Incoming server config message does not have mappings";
    }
    else
        LOG(ERROR) << "Incorrect server config message size received, ret=" << ret << " size="
            << message_size_in_bytes_received;
```

5.2 Printing

5.2.1 Receiving SERVER_DATA Header

```
// First get 4 bytes (FITS_HEADER_LENGTH)
unsigned int size = 0;
memcpy(&size, p_void, FITS_HEADER_LENGTH); // p_void is receive socket buffer

// Get the header based on the size received above
int ret = socket_data_client->Receive(buffer, size);
if (ret > 0 && ret == size)
{
    // Parse the common header
    COMMON_HEADER header;
    header.ParseFromArray(buffer, size);
    if (header.has_tag() && header.has_version() && header.has_message_type() &&
          header.has_message_size_in_bytes())
    {
        TAG tag_received = header.tag();
        VERSION version_received = header.version();
        MESSAGE_TYPE message_type_received = header.message_type();
        unsigned int message_size_in_bytes_received = header.message_size_in_bytes();

        // Now receive the server image data for ths ink
        if (message_size_in_bytes_received != 0 && message_type_received == SERVER_DATA)
            ReceiveServerInkData(message_size_in_bytes_received); // See 5.2.2
        else
            LOG(ERROR) << "Incorrect ink message_size_in_bytes or message_type received";
    }
    else
        LOG(ERROR) << "Incorrect ink header received, does not have all elements";
}
else
    LOG(ERROR) << "Incorrect fixed size ink header received, ret=" << ret << " size=" <<
        size;
```

5.2.2 Receiving SERVER_DATA Ink Data

This is the ReceiveServerInkData shown in 5.2.1

```
int ret = socket_data_client->Recieve(buffer, message_size_in_bytes_received);
if (ret > 0 && ret == message_size_in_bytes_received)
{
    MESSAGE_SERVER_DATA message;
    message.ParseFromArray(buffer, message_size_in_bytes_received);
    if (message.has_job())
    {
        COMMON_JOB job = message.job();
        if (job.has_id() && job.has_image())
        {
            string str_job_id = job.id();
            COMMON_IMAGE image = job.image();
            if (image.has_ink() && image.has_id() && image.has_bits_per_pixel() &&
                image.has_pixels() && image.has_resolution() && image.has_size_in_bytes())
            {
                string ink = image.ink();
                string image_id = image.id();
                unsigned int bits_per_pixel = image.bits_per_pixel();
```

```
                unsigned int bytes_per_scanline = image.bytes_per_scanline();
                COMMON_DIMENSION pixels = image.pixels();
                COMMON_DIMENSION resolution = image.resolution();
                unsigned long long image_size_in_bytes = image.size_in_bytes();

                // Get the image data
                int ret1 = socket_data_client->Recieve(buffer, image_size_in_bytes);
                if (ret1 > 0 && ret1 == image_size_in_bytes)
                {
                    // Print using the data received in the buffer of size image_size_in_bytes

                }
                else
                    LOG(ERROR) << "Incorrect image data size received ret=" << ret << " size="
                      << image_size_in_bytes;
            }
            else
                LOG(ERROR) << "Incoming message does not have an appropriate image object";
        }
        else
            LOG(ERROR) << "Incoming message does not have id or image";
    }
    else
        LOG(ERROR) << "Incoming message does not have job";
}
else
    LOG(ERROR) << "Incorrect server data message size received ret=" << ret << " size=" <<
        message_size_in_bytes_received;
```

6 Package

As indicated earlier, download the protobuf header and library files from https://github.com/protocolbuffers/protobuf/tree/v2.5.0.

You can generate the language specific files from fieryImageTransferServiceIfc.proto file. For C/C++, the following command should be used:

protoc -I=C:/Projects/ifc --cpp_out=C:/Projects/ifc/gen C:/Projects/ifc/fieryImageTransferServiceIfc.proto

This will generate the following 2 files that you should add to your C/C++ project:

- fieryImageTransferSericeIfc.pb.h
- fieryImageTransferService.pb.cc

Although these 2 files can be generated from the proto file, the package provided by Fiery will contain these files for convenience along with the proto file

.

For more information, please visit fiery.com.

Nothing herein should be construed as a warranty in addition to the express warranty statement provided with Fiery, LLC products and services.

ColorGuard, ColorRight, Command WorkStation, ColorWise, Fiery, the Fiery logo, Fiery Compose, Fiery Driven, the Fiery Driven logo, Fiery Edge, Fiery Essential, Fiery HyperRIP, Fiery Impose, Fiery Impress, Fiery ImageViewer, Fiery Intensify, Fiery JobExpert, Fiery JobFlow, Fiery JobMaster, Fiery Prep-it, Fiery Prints, the Fiery Prints logo, Fiery TrueBrand, FreeForm, MicroPress, IQ, PrintMe, RIPChips, RIP-While-Print, Spot-On, Spot Pro, and WebTools are trademarks or registered trademarks of Fiery, LLC and/or its wholly owned subsidiaries in the U.S. and/or certain other countries. All other terms and product names may be trademarks or registered trademarks of their respective owners and are hereby acknowledged.

© 2023 FIERY, LLC. ALL RIGHTS RESERVED.

FTL_XXX.XX.23